

PROMPT TÉCNICO PARA ANTIGRAVITY - MÓDULO 3: DASHBOARD DEL RESTAURANTE

markdown

CONTEXTO DEL MÓDULO 3

Estás construyendo el DASHBOARD DE ADMINISTRACIÓN para restaurantes.

Este es el panel interno donde el restaurante gestiona reservas, mesas, clientes y ve analytics.

Debe ser rápido, intuitivo y funcional para uso diario durante el servicio.

STACK TECNOLÓGICO:

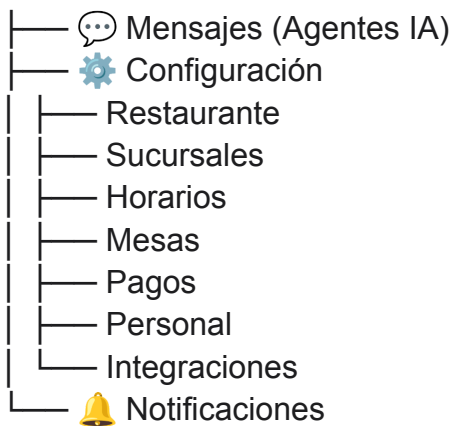
- Frontend: Next.js 14+ (App Router), TypeScript, TailwindCSS
- UI Components: shadcn/ui, Radix UI
- Calendar: react-day-picker + date-fns
- Tables: @tanstack/react-table (tablas avanzadas)
- Charts: recharts o chart.js (gráficos)
- Drag & Drop: @dnd-kit/core (para mesas)
- Forms: React Hook Form + Zod validation
- State Management: Zustand (estado global del dashboard)
- Real-time: Socket.io o Pusher (actualizaciones en tiempo real)
- Export: xlsx (Excel), jsPDF + jspdf-autotable (PDF)
- Date handling: date-fns
- Icons: lucide-react
- Notifications: react-hot-toast

ARQUITECTURA DEL DASHBOARD

ESTRUCTURA DE NAVEGACIÓN:

Sidebar (Lateral izquierdo):

- └─  Inicio (Overview)
- └─  Reservas
 - └─ Calendario
 - └─ Lista
 - └─ Mesas (Floor Plan)
- └─  Clientes (CRM)
- └─  Analytics



****ROLES Y PERMISOS:****

```
``typescript
enum Role {
  OWNER = 'owner',      // Dueño - acceso total
  MANAGER = 'manager',  // Gerente - acceso casi total
  HOSTESS = 'hostess',  // Hostess - solo reservas y mesas
  WAITER = 'waiter',    // Mesero - solo ver reservas asignadas
  VIEWER = 'viewer',    // Solo lectura
}

interface Permissions {
  viewReservations: boolean;
  createReservations: boolean;
  editReservations: boolean;
  deleteReservations: boolean;
  viewCustomers: boolean;
  editCustomers: boolean;
  viewAnalytics: boolean;
  exportData: boolean;
  manageSettings: boolean;
  manageStaff: boolean;
  managePayments: boolean;
}

const rolePermissions: Record<Role, Permissions> = {
  [Role.OWNER]: { /* todo true */ },
  [Role.MANAGER]: { /* casi todo true */ },
  [Role.HOSTESS]: {
    viewReservations: true,
    createReservations: true,
    editReservations: true,
```

```

deleteReservations: true,
viewCustomers: true,
editCustomers: false,
viewAnalytics: false,
exportData: false,
manageSettings: false,
manageStaff: false,
managePayments: false,
},
// ... más roles
};
ESTADO GLOBAL (Zustand Store):
typescript
interface DashboardState {
  // Filtros globales
  selectedBranch: Branch | null;
  selectedDate: Date;
  dateRange: { start: Date; end: Date };

  // Datos en tiempo real
  todayReservations: Booking[];
  activeTables: Table[];
  walkIns: WalkIn[];

  // UI State
  sidebarCollapsed: boolean;
  activeView: 'calendar' | 'list' | 'floor-plan';
  selectedBooking: Booking | null;
  isRealtimeConnected: boolean;

  // Actions
  setSelectedBranch: (branch: Branch) => void;
  setSelectedDate: (date: Date) => void;
  refreshData: () => void;
  // ... más actions
}

```

SECCIÓN 1: OVERVIEW (INICIO)

REQUISITOS FUNCIONALES:

Pantalla principal con métricas clave del día en tiempo real.

CARDS DE MÉTRICAS (Top):

4 cards principales con KPIs del día:

1. Reservas Hoy

- Número total de reservas del día
- Comparación con ayer (↑↓ %)
- Icono: 
- Color: Azul

2. Ocupación Actual

- Porcentaje de mesas ocupadas ahora
- Mesas ocupadas / Total mesas
- Icono: 
- Color: Verde (si <80%), Amarillo (80-95%), Rojo (>95%)

3. Comensales Esperados

- Total de personas que han reservado hoy
- Promedio por reserva
- Icono: 
- Color: Púrpura

4. No-Shows Hoy

- Número de no-shows del día
- Tasa de no-show (%)
- Icono: 
- Color: Rojo si >10%, Verde si <5%

GRÁFICO DE OCUPACIÓN POR HORA:

- Gráfico de líneas/barras mostrando ocupación esperada por hora
- Eje X: Horas del día (12:00 - 23:00)
- Eje Y: Número de reservas o porcentaje de ocupación
- Línea de capacidad máxima (línea roja punteada)
- Barras de reservas confirmadas
- Barras de check-ins realizados
- Hover: muestra detalles de esa hora

LISTA DE PRÓXIMAS RESERVAS:

- Lista de las próximas 10 reservas (ordenadas por hora)
- Cada ítem muestra:
 - Hora
 - Nombre del cliente
 - Número de personas
 - Mesa asignada (si ya está asignada)
 - Estado: Confirmada / Pendiente / Check-in
 - Origen: Web / WhatsApp / Llamada / Walk-in
- Click en reserva → abre modal de detalles
- Botón "Check-in rápido" (marca como llegado)
- Botón "Llamar cliente" (abre tel: link)

ALERTAS Y NOTIFICACIONES:

Panel lateral derecho con alertas en tiempo real:

-  Urgente: "Mesa 5 lleva 2h 30m (tiempo excedido)"
-  Advertencia: "3 reservas pendientes de confirmación"
-  Info: "Juan Pérez acaba de hacer check-in"
-  Sistema: "Nueva reserva por WhatsApp para mañana"

Cada alerta tiene:

- Icono de prioridad
- Timestamp
- Botón de acción (Ver, Marcar como leída, Resolver)

WIDGET DE CLIMA:

- Clima actual de la ciudad del restaurante
- Pronóstico para las próximas horas
- Impacto esperado en reservas (ej: "Lluvia = +15% reservas")

REQUISITOS TÉCNICOS:

Componentes:

typescript

// DashboardOverview.tsx

```
interface DashboardOverviewProps {  
  restaurantId: string;  
  branchId: string;  
}
```

// MetricCard.tsx

```
interface MetricCardProps {  
  title: string;  
  value: number | string;  
  icon: React.ReactNode;  
  trend?: {  
    value: number;  
    isPositive: boolean;  
  };  
  color: 'blue' | 'green' | 'yellow' | 'red' | 'purple';  
}
```

// OccupancyChart.tsx

```
interface OccupancyChartProps {  
  data: Array<{  
    hour: string;
```

```
    expected: number;
    actual: number;
    capacity: number;
  }>;
}
```

```
// UpcomingReservationsList.tsx
interface UpcomingReservationsListProps {
  reservations: Booking[];
  onCheckIn: (bookingId: string) => void;
  onViewDetails: (booking: Booking) => void;
}
```

```
// AlertsPanel.tsx
interface AlertsPanelProps {
  alerts: Alert[];
  onDismiss: (alertId: string) => void;
  onAction: (alertId: string, action: string) => void;
}
```

API Endpoints:

typescript

// GET /api/dashboard/overview

Query params:

- restaurantId: string
- branchId: string
- date: YYYY-MM-DD

```
Response: {
  metrics: {
    totalReservations: number,
    yesterdayReservations: number,
    currentOccupancy: number,
    totalTables: number,
    occupiedTables: number,
    expectedGuests: number,
    avgPartySize: number,
    noShows: number,
    noShowRate: number,
  },
  hourlyData: Array<{
    hour: string,
    expected: number,
    actual: number,
    capacity: number,
```

```

    }>,
    upcomingReservations: Booking[],
    alerts: Alert[],
  }

// GET /api/dashboard/realtime
WebSocket endpoint para actualizaciones en tiempo real
Real-time Updates (Socket.io):
typescript
// server/socket.ts
import { Server } from 'socket.io';

const io = new Server(httpServer, {
  cors: {
    origin: process.env.NEXT_PUBLIC_APP_URL,
  },
});

io.on('connection', (socket) => {
  // Unirse a la sala del restaurante
  socket.on('join-restaurant', (restaurantId: string) => {
    socket.join(`restaurant-${restaurantId}`);
  });

  // Escuchar eventos de reservas
  socket.on('booking-created', (booking: Booking) => {

    io.to(`restaurant-${booking.restaurantId}`).emit('new-booking',
    booking);
  });

  socket.on('booking-checked-in', (bookingId: string) => {
    io.to(`restaurant-${restaurantId}`).emit('check-in',
    bookingId);
  });

  // ... más eventos
});

// client/hooks/useRealtime.ts
import { useEffect } from 'react';
import { io, Socket } from 'socket.io-client';

export function useRealtime(restaurantId: string) {

```

```

useEffect(() => {
  const socket: Socket =
io(process.env.NEXT_PUBLIC_WS_URL!);

  socket.emit('join-restaurant', restaurantId);

  socket.on('new-booking', (booking: Booking) => {
    // Actualizar estado local
    toast.success(`Nueva reserva de
${booking.customer.name}`);
  });

  socket.on('check-in', (bookingId: string) => {
    // Actualizar estado local
  });

  return () => {
    socket.disconnect();
  };
}, [restaurantId]);
}

```

UI/UX:

- Auto-refresh cada 30 segundos (o real-time con WebSockets)
- Skeleton loaders mientras cargan las métricas
- Animaciones suaves al actualizar números (count up/down)
- Tooltips en cada métrica explicando qué significa
- Click en card → navega a vista detallada
- Responsive: cards en grid 2x2 en móvil, 4x1 en desktop
- Dark mode support

SECCIÓN 2: GESTIÓN DE RESERVAS

VISTA 2.1: CALENDARIO

REQUISITOS FUNCIONALES:

Vista de calendario con las reservas del día/semana/mes.

MODOS DE VISTA:

1. Vista Día:

- Timeline vertical de 24 horas (o solo horas de apertura)
- Cada hora dividida en slots de 15/30 min
- Reservas mostradas como bloques de color

- Altura del bloque proporcional a la duración
 - Color según estado:
 - Verde: Confirmada
 - Amarillo: Pendiente
 - Rojo: No-show
 - Gris: Cancelada
 - Click en reserva → modal de detalles
 - Drag & drop para mover reservas (cambia hora)
 - Click en espacio vacío → crear nueva reserva
2. Vista Semana:
- 7 columnas (Lun-Dom)
 - Timeline vertical de horas
 - Reservas como bloques pequeños
 - Hover → tooltip con detalles
 - Click → modal de detalles
3. Vista Mes:
- Calendario tradicional (grid de días)
 - Cada día muestra número de reservas
 - Código de colores por nivel de ocupación
 - Click en día → cambia a vista día

FILTROS:

- Selector de sucursal (si hay varias)
- Selector de fecha (día/semana/mes)
- Filtro por estado (Confirmadas, Pendientes, No-shows, Canceladas)
- Filtro por origen (Web, WhatsApp, Llamada, Walk-in)
- Filtro por tamaño de grupo (1-2, 3-4, 5-6, 7+)
- Búsqueda por nombre o teléfono del cliente

ACCIONES RÁPIDAS:

- Botón "+ Nueva reserva" (abre modal)
- Botón "Exportar" (Excel/PDF)
- Botón "Imprimir" (vista optimizada para impresión)
- Botón "Compartir" (link público del calendario)

MODAL DE DETALLES DE RESERVA:

Al hacer click en una reserva, se abre un modal con:

Header:

- Nombre del cliente
- Estado actual (badge de color)
- Botones de acción rápida:
 -  Check-in

-  Llamar
-  Email
-  WhatsApp
-  Editar
-  Cancelar

Información de la Reserva:

-  Fecha y hora
-  Duración estimada
-  Número de personas
-  Tipo de evento
-  Solicitudes especiales
-  Mesa asignada (o "Sin asignar")
-  Origen de la reserva

Información del Cliente:

- Nombre completo
- Email (con link mailto:)
- Teléfono (con link tel:)
- Historial: "Ha venido X veces" / "Primera vez"
- Notas internas del restaurante
- Rating del cliente (1-5 estrellas)

Timeline de Eventos:

- Lista cronológica de eventos:
 - "Reserva creada por Web - 15/03 10:30"
 - "Email de confirmación enviado - 15/03 10:31"
 - "Recordatorio enviado por WhatsApp - 17/03 18:00"
 - "Check-in realizado - 17/03 20:05"

Pagos (si aplica):

- Monto del depósito
- Estado del pago (Pagado / Pendiente / Reembolsado)
- Método de pago
- Link a transacción en pasarela

Notas Internas:

- Textarea para agregar notas privadas (no visibles al cliente)
- Historial de notas con timestamp y autor

Acciones:

- Check-in: Marca al cliente como llegado, asigna mesa si no está asignada
- No-show: Marca como no-show, aplica penalización si está configurada
- Editar: Cambia fecha/hora/número de personas/mesa
- Mover a otra mesa: Drag & drop o selector
- Extender tiempo: Agrega 30min/1h/1h30m
- Dividir cuenta: (si el POS lo soporta)
- Agregar al CRM: Guarda preferencias del cliente
- Enviar mensaje: WhatsApp/Email/SMS personalizado
- Cancelar: Con opción de reembolso automático

REQUISITOS TÉCNICOS:

Componentes:

typescript

// ReservationsCalendar.tsx

```
interface ReservationsCalendarProps {
  restaurantId: string;
  branchId: string;
  viewMode: 'day' | 'week' | 'month';
  onViewModeChange: (mode: 'day' | 'week' | 'month') => void;
}
```

// DayView.tsx

```
interface DayViewProps {
  date: Date;
  reservations: Booking[];
  onReservationClick: (booking: Booking) => void;
  onTimeSlotClick: (time: string) => void;
  onReservationDrop: (bookingId: string, newTime: string) =>
void;
}
```

// WeekView.tsx

```
interface WeekViewProps {
  startDate: Date;
  reservations: Booking[];
  onReservationClick: (booking: Booking) => void;
}
```

// MonthView.tsx

```
interface MonthViewProps {
  month: Date;
  reservationsByDay: Record<string, Booking[]>;
  onDayClick: (date: Date) => void;
}
```

```
// ReservationModal.tsx
interface ReservationModalProps {
  booking: Booking | null;
  isOpen: boolean;
  onClose: () => void;
  onCheckIn: (bookingId: string) => void;
  onNoShow: (bookingId: string) => void;
  onEdit: (booking: Booking) => void;
  onCancel: (bookingId: string, reason: string) => void;
}
```

```
// ReservationFilters.tsx
interface ReservationFiltersProps {
  filters: ReservationFilters;
  onFilterChange: (filters: ReservationFilters) => void;
}
```

API Endpoints:

typescript

// GET /api/reservations

Query params:

- restaurantId: string
- branchId?: string
- startDate: YYYY-MM-DD
- endDate: YYYY-MM-DD
- status?: string[]
- source?: string[]
- partySize?: string
- search?: string

```
Response: {
  reservations: Booking[],
  total: number,
  page: number,
  pageSize: number,
}
```

// POST /api/reservations

```
Body: {
  restaurantId: string,
  branchId: string,
  date: string,
  time: string,
  partySize: number,
```

```
    customer: {
      name: string,
      email: string,
      phone: string,
    },
    eventType?: string,
    specialRequests?: string,
    tableId?: string,
    notes?: string,
  }
  Response: {
    bookingId: string,
  }
```

```
// PUT /api/reservations/{id}
Body: {
  date?: string,
  time?: string,
  partySize?: number,
  tableId?: string,
  status?: string,
  notes?: string,
}
```

```
// POST /api/reservations/{id}/check-in
Response: {
  success: boolean,
  checkedInAt: string,
}
```

```
// POST /api/reservations/{id}/no-show
Response: {
  success: boolean,
  penaltyApplied: boolean,
  penaltyAmount?: number,
}
```

```
// DELETE /api/reservations/{id}
Body: {
  reason: string,
  refund: boolean,
}
```

Drag & Drop (Calendario):

typescript

```
import { DndContext, DragEndEvent } from '@dnd-kit/core';

function DayView({ reservations, onReservationDrop }) {
  const handleDragEnd = (event: DragEndEvent) => {
    const { active, over } = event;

    if (!over) return;

    const bookingId = active.id as string;
    const newTimeSlot = over.id as string; // ej: "14:30"

    onReservationDrop(bookingId, newTimeSlot);
  };

  return (
    <DndContext onDragEnd={handleDragEnd}>
      {/* Timeline con slots */}
      {reservations.map(booking => (
        <DraggableReservation key={booking.id}
        booking={booking} />
      ))}
    </DndContext>
  );
}
```

UI/UX:

- Timeline con líneas de hora cada 30 min
- Reservas con bordes redondeados y sombras sutiles
- Hover en reserva → elevación y sombra más pronunciada
- Drag preview semi-transparente
- Snap to grid al arrastrar (cada 15 min)
- Animación de transición al mover reservas
- Color coding consistente en todas las vistas
- Tooltips con información clave al hacer hover
- Responsive: vista día en móvil, semana/mes en desktop
- Accesibilidad: keyboard navigation, ARIA labels

VISTA 2.2: LISTA

REQUISITOS FUNCIONALES:

Vista de tabla con todas las reservas, ideal para búsqueda y filtrado avanzado.

COLUMNS DE LA TABLA:

Columna	Contenido	Ancho	Ordenable	Filtrable
Estado	Badge de color (Confirmada, Pendiente, etc.)	100px	✓	✓
Fecha	DD/MM/YYYY	120px	✓	✓
Hora	HH:mm	80px	✓	✗
Cliente	Nombre + Teléfono	200px	✓	✓
Personas	Número + Icono	100px	✓	✓
Mesa	Número de mesa o "Sin asignar"	100px	✗	✗
Origen	Icono (Web, WhatsApp, Llamada, etc.)	100px	✗	✓
Duración	Tiempo estimado	100px	✗	✗
Depósito	Monto pagado o "Sin depósito"	120px	✗	✗
Acciones	Botones (Ver, Editar, Check-in)	150px	✗	✗

FUNCIONALIDADES DE LA TABLA:

- Ordenamiento: Click en header de columna para ordenar ASC/DESC
- Filtrado: Filtros avanzados arriba de la tabla
- Búsqueda: Input de búsqueda global (nombre, teléfono, email)
- Paginación: 20/50/100 registros por página
- Selección múltiple: Checkboxes para acciones masivas
- Acciones masivas:

- Enviar recordatorio a todos
- Exportar selección
- Cancelar selección
- Asignar mesa a selección
- Row actions: Click derecho en fila → menú contextual
- Inline editing: Doble click en celda editable → edita inline
- Expandible: Click en fila → expande para ver más detalles
- Sticky header: Header se mantiene visible al hacer scroll
- Sticky actions: Columna de acciones siempre visible

ESTADOS DE FILA:

- Fila normal: Fondo blanco
- Fila hover: Fondo gris claro
- Fila seleccionada: Fondo azul claro
- Fila de reserva pasada: Opacidad 50%
- Fila de no-show: Fondo rojo claro
- Fila de reserva urgente (próximos 30 min): Borde izquierdo naranja

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// ReservationsList.tsx
interface ReservationsListProps {
  restaurantId: string;
  branchId: string;
}

// ReservationsTable.tsx
interface ReservationsTableProps {
  reservations: Booking[];
  columns: ColumnDef<Booking>[];
  onRowClick: (booking: Booking) => void;
  onBulkAction: (action: string, bookingIds: string[]) =>
void;
}

// ReservationsFilters.tsx
interface ReservationsFiltersProps {
  filters: ReservationFilters;
  onFilterChange: (filters: ReservationFilters) => void;
  onClearFilters: () => void;
}
```



```
// BulkActionsBar.tsx
interface BulkActionsBarProps {
  selectedCount: number;
  onAction: (action: string) => void;
  onClearSelection: () => void;
}
```

@tanstack/react-table Setup:

typescript

```
import {
  useReactTable,
  getCoreRowModel,
  getSortedRowModel,
  getFilteredRowModel,
  getPaginationRowModel,
  ColumnDef,
  flexRender,
} from '@tanstack/react-table';

const columns: ColumnDef<Booking>[] = [
  {
    id: 'select',
    header: ({ table }) => (
      <Checkbox
        checked={table.getIsAllPageRowsSelected()}
        onChange={(value) =>
          table.toggleAllPageRowsSelected(!value)}
      />
    ),
    cell: ({ row }) => (
      <Checkbox
        checked={row.getIsSelected()}
        onChange={(value) =>
          row.toggleSelected(!value)}
      />
    ),
  },
  {
    accessorKey: 'status',
    header: 'Estado',
    cell: ({ row }) => {
      const status = row.original.status;
      return <StatusBadge status={status} />;
    },
  },
];
```

```

    },
  },
  {
    accessorKey: 'date',
    header: 'Fecha',
    cell: ({ row }) => {
      return format(new Date(row.original.date),
'dd/MM/yyyy');
    },
  },
  {
    accessorKey: 'time',
    header: 'Hora',
  },
  {
    accessorKey: 'customer.name',
    header: 'Cliente',
    cell: ({ row }) => {
      const { name, phone } = row.original.customer;
      return (
        <div>
          <div className="font-medium">{name}</div>
          <div className="text-sm text-gray-500">{phone}</div>
        </div>
      );
    },
  },
  {
    accessorKey: 'partySize',
    header: 'Personas',
    cell: ({ row }) => {
      return (
        <div className="flex items-center gap-1">
          <UsersIcon className="w-4 h-4" />
          {row.original.partySize}
        </div>
      );
    },
  },
  // ... más columnas
];

```

```

function ReservationsTable({ reservations }) {
  const table = useReactTable({

```

```

    data: reservations,
    columns,
    getCoreRowModel: getCoreRowModel(),
    getSortedRowModel: getSortedRowModel(),
    getFilteredRowModel: getFilteredRowModel(),
    getPaginationRowModel: getPaginationRowModel(),
    initialState: {
      pagination: { pageSize: 20 },
    },
  });

  return (
    <table>
      <thead>
        {table.getHeaderGroups().map(headerGroup => (
          <tr key={headerGroup.id}>
            {headerGroup.headers.map(header => (
              <th key={header.id}>
                {flexRender(
                  header.column.columnDef.header,
                  header.getContext()
                )}
              </th>
            )}}
          </tr>
        )}}
      </thead>
      <tbody>
        {table.getRowModel().rows.map(row => (
          <tr key={row.id}>
            {row.getVisibleCells().map(cell => (
              <td key={cell.id}>
                {flexRender(cell.column.columnDef.cell,
cell.getContext())}
              </td>
            )}}
          </tr>
        )}}
      </tbody>
    </table>
  );
}

```

Exportación:
typescript

```

import * as XLSX from 'xlsx';
import jsPDF from 'jspdf';
import autoTable from 'jspdf-autotable';

function exportToExcel(reservations: Booking[]) {
  const data = reservations.map(r => ({
    Fecha: format(r.date, 'dd/MM/yyyy'),
    Hora: r.time,
    Cliente: r.customer.name,
    Teléfono: r.customer.phone,
    Email: r.customer.email,
    Personas: r.partySize,
    Mesa: r.table?.number || 'Sin asignar',
    Origen: r.source,
    Estado: r.status,
    Depósito: r.deposit?.amount || 0,
  }));

  const ws = XLSX.utils.json_to_sheet(data);
  const wb = XLSX.utils.book_new();
  XLSX.utils.book_append_sheet(wb, ws, 'Reservas');
  XLSX.writeFile(wb, `reservas-${format(new Date(), 'yyyy-MM-dd')}.xlsx`);
}

function exportToPDF(reservations: Booking[]) {
  const doc = new jsPDF();

  doc.text('Reporte de Reservas', 14, 15);
  doc.text(`Fecha: ${format(new Date(), 'dd/MM/yyyy')}` , 14, 22);

  autoTable(doc, {
    startY: 30,
    head: [['Fecha', 'Hora', 'Cliente', 'Personas', 'Mesa', 'Estado']],
    body: reservations.map(r => [
      format(r.date, 'dd/MM/yyyy'),
      r.time,
      r.customer.name,
      r.partySize,
      r.table?.number || '-',
      r.status,
    ]),
  });
}

```

```
});
```

```
doc.save(`reservas-${format(new Date(),  
'yyyy-MM-dd')} .pdf`);  
}
```

UI/UX:

- Tabla con bordes sutiles y espaciado cómodo
 - Hover en fila → cambio de fondo
 - Checkboxes con animación de check
 - Badges de estado con colores consistentes
 - Iconos para origen (Web, WhatsApp, etc.)
 - Paginación con selector de página size
 - Filtros colapsables (show/hide)
 - Loading skeleton mientras cargan datos
 - Empty state si no hay reservas
 - Responsive: scroll horizontal en móvil, columnas ocultas
-

VISTA 2.3: FLOOR PLAN (PLANO DE MESAS)

REQUISITOS FUNCIONALES:

Vista visual del restaurante con las mesas en su ubicación real.

CARACTERÍSTICAS DEL PLANO:

- Canvas interactivo: Área donde se dibujan las mesas
- Mesas como elementos visuales:
 - Formas: Circular, cuadrada, rectangular
 - Tamaño proporcional a capacidad
 - Color según estado:
 - Verde: Disponible
 - Amarillo: Reservada (próximamente)
 - Rojo: Ocupada
 - Gris: Fuera de servicio
 - Número de mesa visible
 - Número de personas (si está ocupada)
 - Hora de reserva (si está reservada)
- Zonas del restaurante:
 - Interior
 - Terraza
 - Bar
 - Salón privado
 - Cada zona con nombre y color de fondo sutil
- Drag & Drop:

- Mover reservas de una mesa a otra
- Mover mesas de posición (solo para administradores)
- Redimensionar mesas (cambiar capacidad)
- Click en mesa:
 - Si está disponible → modal para crear reserva
 - Si está reservada → modal de detalles de reserva
 - Si está ocupada → opciones (check-out, extender tiempo, etc.)

LEYENDA:

Panel lateral con leyenda de colores y estados:

-  Disponible
-  Reservada
-  Ocupada
-  Fuera de servicio
-  VIP

CONTROLES:

- Zoom in/out (scroll o botones)
- Pan (arrastrar el canvas)
- Reset view (volver a vista original)
- Toggle zonas (mostrar/ocultar zonas)
- Toggle números de mesa
- Modo edición (mover mesas) vs Modo operación (solo ver)

ESTADÍSTICAS EN TIEMPO REAL:

Panel superior con métricas del floor plan:

- Mesas disponibles: X / Total
- Mesas ocupadas: X / Total
- Tiempo promedio de ocupación: X min
- Próxima mesa disponible: Mesa X en Y min

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// FloorPlan.tsx
interface FloorPlanProps {
  restaurantId: string;
  branchId: string;
  tables: Table[];
  reservations: Booking[];
  onTableClick: (table: Table) => void;
```

```

    onReservationDrop: (bookingId: string, tableId: string) =>
void;
}

```

```

// TableElement.tsx
interface TableElementProps {
    table: Table;
    reservation?: Booking;
    status: 'available' | 'reserved' | 'occupied' |
'out-of-service';
    onClick: () => void;
    onDrop: (bookingId: string) => void;
}

```

```

// ZoneElement.tsx
interface ZoneElementProps {
    zone: Zone;
    tables: Table[];
}

```

```

// FloorPlanControls.tsx
interface FloorPlanControlsProps {
    onZoomIn: () => void;
    onZoomOut: () => void;
    onResetView: () => void;
    editMode: boolean;
    onEditModeToggle: () => void;
}

```

```

// FloorPlanLegend.tsx
interface FloorPlanLegendProps {
    stats: {
        available: number;
        reserved: number;
        occupied: number;
        outOfService: number;
    };
}

```

Canvas con React Konva o [Fabric.js](#):

```

typescript
import { Stage, Layer, Rect, Circle, Text } from
'react-konva';

```

```

function FloorPlan({ tables, reservations }) {

```

```

const [scale, setScale] = useState(1);
const [position, setPosition] = useState({ x: 0, y: 0 });

return (
  <Stage
    width={window.innerWidth}
    height={window.innerHeight}
    scaleX={scale}
    scaleY={scale}
    x={position.x}
    y={position.y}
    onWheel={handleWheel} // zoom
    onDragEnd={handleDragEnd} // pan
  >
    <Layer>
      { /* Zonas */ }
      { zones.map(zone => (
        <Rect
          key={zone.id}
          x={zone.x}
          y={zone.y}
          width={zone.width}
          height={zone.height}
          fill={zone.color}
          opacity={0.2}
        />
      )) }

      { /* Mesas */ }
      { tables.map(table => {
        const reservation = reservations.find(r => r.tableId
=== table.id);
        const status = getTableStatus(table, reservation);

        return table.shape === 'circle' ? (
          <Circle
            key={table.id}
            x={table.x}
            y={table.y}
            radius={table.radius}
            fill={getStatusColor(status)}
            draggable={editMode}
            onClick={() => onTableClick(table)}

```



```

        onDragEnd={ (e) => handleTableDragEnd(table.id,
e) }
      />
    ) : (
      <Rect
        key={table.id}
        x={table.x}
        y={table.y}
        width={table.width}
        height={table.height}
        fill={getStatusColor(status)}
        draggable={editMode}
        onClick={() => onTableClick(table)}
        onDragEnd={ (e) => handleTableDragEnd(table.id,
e) }
      />
    );
  })}

```

```

    { /* Números de mesa */ }
    { tables.map(table => (
      <Text
        key={`text-${table.id}`}
        x={table.x}
        y={table.y}
        text={table.number.toString()}
        fontSize={16}
        fill="white"
        align="center"
        verticalAlign="middle"
      />
    )) }
  </Layer>
</Stage>
);
}

```

API Endpoints:

typescript

// GET /api/floor-plan

Query params:

- branchId: string

Response: {

 zones: Zone[],

```
    tables: Table[],  
}
```

```
// PUT /api/tables/{id}
```

```
Body: {  
  x?: number,  
  y?: number,  
  width?: number,  
  height?: number,  
  capacity?: number,  
  zoneId?: string,  
}
```

```
// POST /api/floor-plan/zones
```

```
Body: {  
  name: string,  
  x: number,  
  y: number,  
  width: number,  
  height: number,  
  color: string,  
}
```

Database Schema:

prisma

```
model Table {  
  id          String    @id @default(uuid())  
  branchId    String  
  number      Int  
  shape       String    // circle, square, rectangle  
  x           Float  
  y           Float  
  width       Float?    // para square/rectangle  
  height      Float?    // para square/rectangle  
  radius      Float?    // para circle  
  capacity    Int  
  zoneId      String?  
  isActive    Boolean   @default(true)  
  createdAt   DateTime  @default(now())  
  updatedAt   DateTime  @updatedAt  
  
  branch      Branch    @relation(fields: [branchId],  
references: [id])  
  zone        Zone?     @relation(fields: [zoneId], references:  
[id])
```

```

    reservations Booking[]

    @@unique([branchId, number])
}

model Zone {
    id          String    @id @default(uuid())
    branchId    String
    name        String
    x           Float
    y           Float
    width       Float
    height      Float
    color       String
    createdAt   DateTime @default(now())

    branch      Branch    @relation(fields: [branchId], references:
[id])
    tables      Table[]
}

```

UI/UX:

- Canvas con fondo de cuadrícula sutil
- Mesas con sombras para dar profundidad
- Animación de transición al cambiar estado
- Tooltip al hacer hover en mesa (muestra detalles)
- Drag preview semi-transparente
- Snap to grid al mover mesas (cada 10px)
- Zoom suave con scroll
- Indicador de zoom actual (ej: 100%)
- Minimap en esquina (vista general del plano)
- Responsive: scroll y zoom en móvil, controles completos en desktop

SECCIÓN 3: CLIENTES (CRM)

REQUISITOS FUNCIONALES:

Sistema de gestión de clientes (CRM básico) para conocer y fidelizar a los comensales.

LISTA DE CLIENTES:

Tabla con todos los clientes que han reservado alguna vez.

COLUMNAS:

Columna	Contenido	Ordenable	Filtrable
Nombre	Nombre completo	✓	✓
Email	Email	✓	✓
Teléfono	Teléfono	✗	✓
Visitas	Número de visitas	✓	✓
Última visita	Fecha de última reserva	✓	✗
Gasto total	Suma de depósitos pagados	✓	✗
Rating	Promedio de rating (1-5)	✓	✗
Segmento	VIP / Regular / Nuevo / Inactivo	✗	✓
Acciones	Ver perfil, Enviar mensaje	✗	✗

FILTROS AVANZADOS:

- Búsqueda por nombre, email o teléfono
- Filtro por segmento (VIP, Regular, Nuevo, Inactivo)
- Filtro por número de visitas (1, 2-5, 6-10, 10+)
- Filtro por fecha de última visita (último mes, 3 meses, 6 meses, +1 año)
- Filtro por origen de reservas (Web, WhatsApp, Llamada)
- Filtro por gasto total (rangos personalizados)

PERFIL DEL CLIENTE:

Al hacer click en un cliente, se abre su perfil completo:

Header:

- Foto de perfil (si tiene, o avatar generado con iniciales)
- Nombre completo

- Email y teléfono (con links)
- Segmento (badge de color)
- Fecha de primera visita
- Total de visitas

Tabs:

1. Resumen:

- Métricas clave:
 - Total de visitas
 - Gasto total
 - Promedio de personas por reserva
 - Promedio de gasto por visita
 - Días desde última visita
- Gráfico de visitas por mes (últimos 12 meses)
- Tipo de evento más frecuente
- Día de la semana preferido
- Hora preferida

2. Historial de Reservas:

- Lista de todas las reservas del cliente
- Cada reserva muestra:
 - Fecha y hora
 - Número de personas
 - Mesa asignada
 - Estado (Completada, No-show, Cancelada)
 - Rating dado (si aplica)
 - Notas internas
- Click en reserva → abre detalles

3. Preferencias:

- Tipo de evento favorito
- Solicitudes especiales frecuentes (ej: "Siempre pide silla alta")
- Alergias o restricciones alimentarias
- Bebidas preferidas
- Platos favoritos (si el POS integra)
- Mesa preferida
- Notas del restaurante (textarea editable)

4. Comunicaciones:

- Historial de emails enviados
- Historial de WhatsApps enviados
- Historial de SMS enviados
- Preferencias de comunicación:
 - Email (Sí/No)
 - WhatsApp (Sí/No)
 - SMS (Sí/No)

- Promociones (Sí/No)
 - Fecha de última comunicación
 - Botón "Enviar mensaje" (abre modal)
- 5. Feedback:
 - Lista de reseñas/feedback dado por el cliente
 - Rating promedio
 - Comentarios textuales
 - Fecha de cada feedback
- 6. Etiquetas:
 - Etiquetas personalizadas (ej: "Celíaco", "Cliente corporativo", "Influencer")
 - Crear nueva etiqueta
 - Agregar/quitar etiquetas

SEGMENTACIÓN AUTOMÁTICA:

El sistema clasifica automáticamente a los clientes en segmentos:

- VIP: +10 visitas O +\$500.000 COP gastado
- Regular: 3-10 visitas
- Nuevo: 1-2 visitas
- Inactivo: No ha venido en +6 meses
- En riesgo: No ha venido en +3 meses (pero era regular)
- Alto valor: Gasto promedio por visita >\$100.000 COP

ACCIONES MASIVAS:

- Seleccionar múltiples clientes → acciones:
 - Enviar email masivo
 - Enviar WhatsApp masivo
 - Agregar etiqueta
 - Cambiar segmento manualmente
 - Exportar a Excel/CSV

EXPORTACIÓN:

- Exportar lista de clientes a Excel/CSV
- Exportar segmento específico
- Incluir/excluir campos personalizados

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// CustomersList.tsx
interface CustomersListProps {
```

```
    restaurantId: string;
}

// CustomerProfile.tsx
interface CustomerProfileProps {
    customerId: string;
}

// CustomerSummary.tsx
interface CustomerSummaryProps {
    customer: Customer;
    stats: CustomerStats;
}

// CustomerReservationsHistory.tsx
interface CustomerReservationsHistoryProps {
    customerId: string;
}

// CustomerPreferences.tsx
interface CustomerPreferencesProps {
    customerId: string;
    preferences: CustomerPreferences;
    onUpdate: (preferences: CustomerPreferences) => void;
}

// CustomerCommunications.tsx
interface CustomerCommunicationsProps {
    customerId: string;
    communications: Communication[];
}

// CustomerTags.tsx
interface CustomerTagsProps {
    customerId: string;
    tags: string[];
    onAddTag: (tag: string) => void;
    onRemoveTag: (tag: string) => void;
}

// CustomerFilters.tsx
interface CustomerFiltersProps {
    filters: CustomerFilters;
    onFilterChange: (filters: CustomerFilters) => void;
}
```

```
}
```

API Endpoints:

```
typescript
```

```
// GET /api/customers
```

Query params:

- restaurantId: string
- search?: string
- segment?: string
- minVisits?: number
- maxVisits?: number
- lastVisitAfter?: date
- lastVisitBefore?: date
- page?: number
- pageSize?: number

Response: {

```
  customers: Customer[],
  total: number,
  page: number,
  pageSize: number,
```

```
}
```

```
// GET /api/customers/{id}
```

Response: {

```
  customer: Customer,
  stats: CustomerStats,
  preferences: CustomerPreferences,
  communications: Communication[],
  tags: string[],
```

```
}
```

```
// PUT /api/customers/{id}
```

Body: {

```
  name?: string,
  email?: string,
  phone?: string,
  preferences?: CustomerPreferences,
  notes?: string,
  tags?: string[],
```

```
}
```

```
// POST /api/customers/{id}/communications
```

Body: {

```
  type: 'email' | 'whatsapp' | 'sms',
```



```
    message: string,  
    sentAt: string,  
}
```

```
// POST /api/customers/bulk-action
```

```
Body: {  
  customerIds: string[],  
  action: 'send-email' | 'send-whatsapp' | 'add-tag' |  
  'remove-tag' | 'export',  
  payload?: any,  
}
```

Database Schema:

```
prisma  
model Customer {  
  id          String    @id @default(uuid())  
  restaurantId String  
  name        String  
  email       String?  
  phone       String?  
  segment     String    @default("new") // vip, regular,  
new, inactive, at-risk  
  totalVisits Int        @default(0)  
  totalSpent   Float      @default(0)  
  avgPartySize Float      @default(0)  
  avgSpendPerVisit Float  @default(0)  
  firstVisitAt DateTime?  
  lastVisitAt  DateTime?  
  notes        String?  
  createdAt    DateTime  @default(now())  
  updatedAt    DateTime  @updatedAt  
  
  restaurant Restaurant @relation(fields: [restaurantId],  
references: [id])  
  reservations Booking[]  
  preferences CustomerPreferences?  
  communications Communication[]  
  tags          CustomerTag[]  
  feedback      Feedback[]  
  
  @@unique([restaurantId, email])  
}
```

```
model CustomerPreferences {  
  id          String    @id @default(uuid())
```

```

    customerId      String    @unique
    favoriteEventType String?
    frequentRequests String[] // array de strings
    allergies        String[]
    favoriteDrinks    String[]
    favoriteDishes    String[]
    preferredTableId  String?
    preferredDayOfWeek Int?    // 0-6
    preferredTime      String? // "19:00"
    contactEmail       Boolean  @default(true)
    contactWhatsApp    Boolean  @default(true)
    contactSMS         Boolean  @default(false)
    contactPromotions  Boolean  @default(false)

    customer Customer @relation(fields: [customerId],
references: [id], onDelete: Cascade)
}

model CustomerTag {
    id          String    @id @default(uuid())
    customerId  String
    tag         String
    createdAt   DateTime @default(now())

    customer Customer @relation(fields: [customerId],
references: [id], onDelete: Cascade)

    @@unique([customerId, tag])
}

model Communication {
    id          String    @id @default(uuid())
    customerId  String
    type        String    // email, whatsapp, sms
    message     String
    sentAt      DateTime
    readAt      DateTime?

    customer Customer @relation(fields: [customerId],
references: [id], onDelete: Cascade)
}

```

Segmentación Automática:

```

typescript
// Cron job que se ejecuta diariamente

```

```

async function updateCustomerSegments(restaurantId: string) {
  const customers = await prisma.customer.findMany({
    where: { restaurantId },
    include: { reservations: true },
  });

  for (const customer of customers) {
    let segment = 'new';

    const daysSinceLastVisit = customer.lastVisitAt
      ? differenceInDays(new Date(), customer.lastVisitAt)
      : Infinity;

    if (customer.totalVisits >= 10 || customer.totalSpent >=
500000) {
      segment = 'vip';
    } else if (customer.totalVisits >= 3) {
      segment = 'regular';
    } else if (customer.totalVisits >= 1) {
      segment = 'new';
    }

    // Check for at-risk
    if (segment === 'regular' && daysSinceLastVisit > 90) {
      segment = 'at-risk';
    }

    // Check for inactive
    if (daysSinceLastVisit > 180) {
      segment = 'inactive';
    }

    await prisma.customer.update({
      where: { id: customer.id },
      data: { segment },
    });
  }
}

```

UI/UX:

- Tabla con diseño limpio y espaciado cómodo
- Avatares con iniciales si no hay foto
- Badges de segmento con colores distintivos
- Iconos para acciones rápidas

- Modal de perfil con tabs claras
 - Gráficos interactivos (hover para detalles)
 - Timeline visual del historial
 - Empty states amigables
 - Loading skeletons
 - Responsive: tabla con scroll horizontal en móvil
-

SECCIÓN 4: ANALYTICS

REQUISITOS FUNCIONALES:

Dashboard de analytics con métricas clave y gráficos para tomar decisiones de negocio.

MÉTRICAS PRINCIPALES (KPIs):

1. Reservas:
 - Total de reservas (día/semana/mes/año)
 - Tasa de crecimiento vs período anterior
 - Reservas por canal de origen (Web, WhatsApp, Llamada, Walk-in)
 - Tasa de conversión (visitantes web → reservas)
2. Ocupación:
 - Porcentaje de ocupación promedio
 - Picos de ocupación (días/horas)
 - Mesas más/menos populares
 - Tiempo promedio de rotación de mesa
3. Clientes:
 - Total de clientes únicos
 - Nuevos clientes vs recurrentes
 - Tasa de retención (% que vuelve)
 - Valor promedio por cliente (LTV)
4. No-Shows:
 - Tasa de no-shows (%)
 - No-shows por canal de origen
 - No-shows por día de la semana
 - Ingresos perdidos por no-shows
5. Ingresos:
 - Total de depósitos cobrados
 - Ingresos estimados (basado en reservas)
 - Ticket promedio por reserva
 - Ticket promedio por persona

GRÁFICOS:

1. Reservas por Día (Línea):
 - Eje X: Días (últimos 30 días)
 - Eje Y: Número de reservas
 - Línea de reservas totales
 - Línea de no-shows
 - Hover: muestra detalles del día
2. Ocupación por Hora (Barras):
 - Eje X: Horas del día (12:00 - 23:00)
 - Eje Y: Porcentaje de ocupación
 - Barras de color según nivel (verde/amarillo/rojo)
 - Línea de capacidad máxima
3. Origen de Reservas (Donut/Pie):
 - Porcentaje de reservas por canal
 - Web, WhatsApp, Llamada, Walk-in, Instagram, etc.
 - Click en segmento → filtra datos
4. Distribución de Tamaño de Grupos (Barras):
 - Eje X: Tamaño (1, 2, 3, 4, 5, 6, 7+)
 - Eje Y: Número de reservas
 - Útil para optimizar configuración de mesas
5. Día de la Semana Más Popular (Barras Horizontales):
 - Eje X: Número de reservas
 - Eje Y: Días (Lun-Dom)
 - Ordenado de mayor a menor
6. Tendencia de Clientes Nuevos vs Recurrentes (Área Apilada):
 - Eje X: Semanas/Meses
 - Eje Y: Número de clientes
 - Área verde: Clientes nuevos
 - Área azul: Clientes recurrentes
7. Heatmap de Ocupación (Calendario):
 - Calendario de 30 días
 - Color de cada día según ocupación (verde bajo → rojo alto)
 - Click en día → muestra detalles

FILTROS:

- Rango de fechas (últimos 7 días, 30 días, 3 meses, 6 meses, 1 año, personalizado)
- Sucursal (si hay varias)
- Comparar con período anterior (Sí/No)
- Tipo de evento (Cena romántica, Cumpleaños, etc.)

EXPORTACIÓN:

- Exportar reporte completo a PDF
- Exportar datos a Excel/CSV

- Programar reportes automáticos por email (diario/semanal/mensual)

INSIGHTS AUTOMÁTICOS:

El sistema genera insights automáticos basados en los datos:

- "📈 Tus reservas aumentaron 23% esta semana vs la anterior"
- "⚠️ Tu tasa de no-shows subió a 15% (promedio industry: 8%)"
- "💡 Los viernes a las 20:00 son tu horario más popular"
- "🎯 El 40% de tus reservas son para 2 personas"
- "📱 El 65% de tus reservas vienen por WhatsApp"
- "💰 Tu ticket promedio es \$80.000 COP por reserva"

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// AnalyticsDashboard.tsx
interface AnalyticsDashboardProps {
  restaurantId: string;
  branchId?: string;
}

// KPICards.tsx
interface KPICardsProps {
  metrics: AnalyticsMetrics;
  dateRange: DateRange;
}

// ReservationsChart.tsx
interface ReservationsChartProps {
  data: Array<{
    date: string;
    total: number;
    noShows: number;
  }>;
}

// OccupancyByHourChart.tsx
interface OccupancyByHourChartProps {
  data: Array<{
    hour: string;
    occupancy: number;
    capacity: number;
  }>;
}
```

```
}
```

```
// SourceDistributionChart.tsx
```

```
interface SourceDistributionChartProps {  
  data: Array<{  
    source: string;  
    count: number;  
    percentage: number;  
  }>;  
}
```

```
// PartySizeDistributionChart.tsx
```

```
interface PartySizeDistributionChartProps {  
  data: Array<{  
    size: number;  
    count: number;  
  }>;  
}
```

```
// DayOfWeekChart.tsx
```

```
interface DayOfWeekChartProps {  
  data: Array<{  
    day: string;  
    count: number;  
  }>;  
}
```

```
// NewVsReturningChart.tsx
```

```
interface NewVsReturningChartProps {  
  data: Array<{  
    date: string;  
    new: number;  
    returning: number;  
  }>;  
}
```

```
// OccupancyHeatmap.tsx
```

```
interface OccupancyHeatmapProps {  
  data: Array<{  
    date: string;  
    occupancy: number;  
  }>;  
}
```

```
// InsightsPanel.tsx
interface InsightsPanelProps {
  insights: Insight[];
}

// AnalyticsFilters.tsx
interface AnalyticsFiltersProps {
  dateRange: DateRange;
  branchId?: string;
  onFilterChange: (filters: AnalyticsFilters) => void;
}
```

Recharts Implementation:

```
typescript
import {
  LineChart,
  Line,
  BarChart,
  Bar,
  PieChart,
  Pie,
  Cell,
  XAxis,
  YAxis,
  CartesianGrid,
  Tooltip,
  Legend,
  ResponsiveContainer,
} from 'recharts';

function ReservationsChart({ data }) {
  return (
    <ResponsiveContainer width="100%" height={300}>
      <LineChart data={data}>
        <CartesianGrid strokeDasharray="3 3" />
        <XAxis dataKey="date" />
        <YAxis />
        <Tooltip />
        <Legend />
        <Line
          type="monotone"
          dataKey="total"
          stroke="#3b82f6"
          strokeWidth={2}
          name="Total Reservas"
        />
      </LineChart>
    </ResponsiveContainer>
  );
}
```



```

        />
        <Line
            type="monotone"
            dataKey="noShows"
            stroke="#ef4444"
            strokeWidth={2}
            name="No-Shows"
        />
    </LineChart>
</ResponsiveContainer>
);
}

```

```

function SourceDistributionChart({ data }) {
    const COLORS = ['#3b82f6', '#10b981', '#f59e0b', '#ef4444',
    '#8b5cf6'];

```

```

    return (
        <ResponsiveContainer width="100%" height={300}>
            <PieChart>
                <Pie
                    data={data}
                    cx="50%"
                    cy="50%"
                    labelLine={false}
                    label={({ source, percentage }) => `${source}:
${percentage}%`}
                    outerRadius={100}
                    fill="#8884d8"
                    dataKey="count"
                >
                    {data.map((entry, index) => (
                        <Cell key={`cell-${index}`} fill={COLORS[index %
COLORS.length]} />
                    ))}
                </Pie>
                <Tooltip />
            </PieChart>
        </ResponsiveContainer>
    );
}

```

API Endpoints:

```

typescript
// GET /api/analytics/overview

```

Query params:

- restaurantId: string
- branchId?: string
- startDate: YYYY-MM-DD
- endDate: YYYY-MM-DD

Response: {

```
  metrics: {
    totalReservations: number,
    reservationGrowth: number, // percentage
    reservationsBySource: Array<{source: string, count:
number}>,
    conversionRate: number,
    avgOccupancy: number,
    peakHours: Array<{hour: string, occupancy: number}>,
    popularTables: Array<{tableId: string, count: number}>,
    avgRotationTime: number, // minutes
    totalCustomers: number,
    newCustomers: number,
    returningCustomers: number,
    retentionRate: number,
    customerLTV: number,
    noShowRate: number,
    noShowsBySource: Array<{source: string, count: number}>,
    lostRevenue: number,
    totalDeposits: number,
    estimatedRevenue: number,
    avgTicketPerReservation: number,
    avgTicketPerPerson: number,
  },
  chartsData: {
    reservationsByDay: Array<{date: string, total: number,
noShows: number}>,
    occupancyByHour: Array<{hour: string, occupancy: number,
capacity: number}>,
    sourceDistribution: Array<{source: string, count: number,
percentage: number}>,
    partySizeDistribution: Array<{size: number, count:
number}>,
    dayOfWeekDistribution: Array<{day: string, count:
number}>,
    newVsReturning: Array<{date: string, new: number,
returning: number}>,
```

```

    occupancyHeatmap: Array<{date: string, occupancy:
number}>,
  },
  insights: Array<{
    type: 'positive' | 'warning' | 'suggestion',
    message: string,
    metric?: string,
    value?: number,
  }>,
}

```

// GET /api/analytics/export

Query params:

```

- format: 'pdf' | 'excel' | 'csv'
- startDate: YYYY-MM-DD
- endDate: YYYY-MM-DD

```

Response: File (download)

Generación de Insights:

typescript

```
function generateInsights(metrics: AnalyticsMetrics):
```

```
Insight[] {
```

```
  const insights: Insight[] = [];
```

```
  // Reservation growth
```

```
  if (metrics.reservationGrowth > 10) {
```

```
    insights.push({
```

```
      type: 'positive',
```

```
      message: `📈 Tus reservas aumentaron
${metrics.reservationGrowth}% este período`,
```

```
      metric: 'reservationGrowth',
```

```
      value: metrics.reservationGrowth,
```

```
    });
```

```
  }
```

```
  // No-show rate
```

```
  if (metrics.noShowRate > 10) {
```

```
    insights.push({
```

```
      type: 'warning',
```

```
      message: `⚠️ Tu tasa de no-shows es
${metrics.noShowRate}% (promedio industry: 8%)`,
```

```
      metric: 'noShowRate',
```

```
      value: metrics.noShowRate,
```

```
    });
```

```

    }

    // Popular day
    const popularDay = metrics.dayOfWeekDistribution.sort((a, b)
=> b.count - a.count)[0];
    insights.push({
      type: 'suggestion',
      message: `💡 ${popularDay.day} es tu día más popular con
${popularDay.count} reservas`,
      metric: 'popularDay',
      value: popularDay.count,
    });

    // Source distribution
    const topSource = metrics.reservationsBySource.sort((a, b)
=> b.count - a.count)[0];
    insights.push({
      type: 'suggestion',
      message: `📱 El ${Math.round(topSource.count /
metrics.totalReservations * 100)}% de tus reservas vienen por
${topSource.source}`,
      metric: 'topSource',
      value: topSource.count,
    });

    return insights;
  }
}

```

UI/UX:

- Cards de KPIs con iconos y colores distintivos
- Gráficos interactivos (hover, click, zoom)
- Tooltips detallados en gráficos
- Leyendas claras
- Animaciones suaves al cargar datos
- Skeleton loaders mientras cargan
- Empty states si no hay datos suficientes
- Responsive: gráficos se adaptan al tamaño de pantalla
- Dark mode support
- Accesibilidad: ARIA labels en gráficos

SECCIÓN 5: CONFIGURACIÓN

REQUISITOS FUNCIONALES:

Panel de configuración para gestionar todos los aspectos del restaurante.

SUBSECCIONES:

1. Restaurante:

- Información básica (nombre, dirección, teléfono, email)
- Descripción
- Redes sociales
- Logo e imágenes
- Colores y branding

2. Sucursales:

- Lista de sucursales
- Crear nueva sucursal
- Editar sucursal existente
- Eliminar sucursal
- Configurar horarios y capacidad por sucursal

3. Horarios:

- Horarios de apertura por día de la semana
- Múltiples turnos por día (almuerzo, cena)
- Horarios especiales (festivos, eventos)
- Bloqueo de fechas (vacaciones, mantenimiento)

4. Mesas:

- Configuración de tipos de mesa
- Capacidad de cada tipo
- Cantidad de mesas por tipo
- Floor plan editor (ver Sección 2.3)

5. Pagos:

- Configurar depósitos (sí/no, porcentaje, monto fijo)
- Pasarelas de pago (Wompi, Stripe, PayU, ePayco)
- Credenciales de pasarelas
- Políticas de reembolso
- Cobro por no-show

6. Personal:

- Lista de usuarios con acceso al dashboard
- Crear nuevo usuario
- Asignar roles y permisos
- Eliminar usuario
- Ver actividad de usuarios (logs)

7. Integraciones:

- Conectar con POS (Vendty, SoftRestaurant, etc.)
- Conectar con Google Business
- Conectar con Google Calendar
- Webhooks configurables
- API keys para desarrolladores

8. Notificaciones:

- Configurar emails de confirmación
- Configurar recordatorios (WhatsApp, SMS, Email)
- Timing de recordatorios (24h antes, 2h antes)
- Templates de mensajes personalizables

9. Políticas:

- Política de cancelación
- Política de no-shows
- Términos y condiciones
- Política de privacidad

10. Avanzado:

- Dominio personalizado
- Widget embebible
- Exportar/importar datos
- Logs de auditoría
- Backup de datos

REQUISITOS TÉCNICOS:

Componentes:

typescript

// SettingsPage.tsx

```
interface SettingsPageProps {  
  restaurantId: string;  
}
```

// RestaurantSettings.tsx

```
interface RestaurantSettingsProps {  
  restaurant: Restaurant;  
  onUpdate: (data: Partial<Restaurant>) => void;  
}
```

// BranchesSettings.tsx

```
interface BranchesSettingsProps {  
  branches: Branch[];  
  onCreateBranch: (data: CreateBranchData) => void;  
  onUpdateBranch: (id: string, data: Partial<Branch>) => void;  
  onDeleteBranch: (id: string) => void;  
}
```

// SchedulesSettings.tsx

```
interface SchedulesSettingsProps {  
  schedules: Schedule[];  
  onUpdate: (schedules: Schedule[]) => void;  
}
```

```

// TablesSettings.tsx
interface TablesSettingsProps {
  tableTypes: TableType[];
  onUpdate: (tableTypes: TableType[]) => void;
}

// PaymentSettings.tsx
interface PaymentSettingsProps {
  paymentSettings: PaymentSettings;
  onUpdate: (data: Partial<PaymentSettings>) => void;
}

// StaffSettings.tsx
interface StaffSettingsProps {
  staff: StaffMember[];
  onCreateStaff: (data: CreateStaffData) => void;
  onUpdateStaff: (id: string, data: Partial<StaffMember>) =>
void;
  onDeleteStaff: (id: string) => void;
}

// IntegrationsSettings.tsx
interface IntegrationsSettingsProps {
  integrations: Integration[];
  onConnect: (integrationId: string, credentials: any) =>
void;
  onDisconnect: (integrationId: string) => void;
}

// NotificationsSettings.tsx
interface NotificationsSettingsProps {
  settings: NotificationSettings;
  onUpdate: (settings: NotificationSettings) => void;
}

// PoliciesSettings.tsx
interface PoliciesSettingsProps {
  policies: Policies;
  onUpdate: (policies: Policies) => void;
}

```

API Endpoints:

```

typescript
// GET /api/settings/restaurant

```

Response: Restaurant

// PUT /api/settings/restaurant

Body: Partial<Restaurant>

Response: Restaurant

// GET /api/settings/branches

Response: Branch[]

// POST /api/settings/branches

Body: CreateBranchData

Response: Branch

// PUT /api/settings/branches/{id}

Body: Partial<Branch>

Response: Branch

// DELETE /api/settings/branches/{id}

Response: { success: boolean }

// GET /api/settings/schedules

Response: Schedule[]

// PUT /api/settings/schedules

Body: Schedule[]

Response: Schedule[]

// GET /api/settings/payment

Response: PaymentSettings

// PUT /api/settings/payment

Body: Partial<PaymentSettings>

Response: PaymentSettings

// GET /api/settings/staff

Response: StaffMember[]

// POST /api/settings/staff

Body: CreateStaffData

Response: StaffMember

// PUT /api/settings/staff/{id}

Body: Partial<StaffMember>

Response: StaffMember


```
// DELETE /api/settings/staff/{id}
Response: { success: boolean }
```

```
// GET /api/settings/integrations
Response: Integration[]
```

```
// POST /api/settings/integrations/{id}/connect
Body: { credentials: any }
Response: Integration
```

```
// POST /api/settings/integrations/{id}/disconnect
Response: { success: boolean }
```

UI/UX:

- Sidebar de navegación con las subsecciones
- Formularios con validación en tiempo real
- Botones "Guardar" y "Cancelar" en cada sección
- Confirmación antes de eliminar datos críticos
- Tooltips explicativos en campos complejos
- Preview en vivo de cambios (ej: colores, logo)
- Loading states al guardar
- Mensajes de éxito/error claros
- Responsive: sidebar colapsable en móvil
- Accesibilidad: ARIA labels, keyboard navigation

SECCIÓN 6: MENSAJES (AGENTES IA)

REQUISITOS FUNCIONALES:

Panel para ver y gestionar las conversaciones de los agentes de IA (WhatsApp, chat web, etc.).

LISTA DE CONVERSACIONES:

- Lista de todas las conversaciones activas y recientes
- Cada conversación muestra:
 - Avatar del cliente (o iniciales)
 - Nombre del cliente
 - Canal (WhatsApp, Web Chat, Instagram, etc.)
 - Último mensaje (preview)
 - Timestamp del último mensaje
 - Badge de no leídos
 - Estado: Activa / Resuelta / Bot

- Ordenamiento: Más reciente primero
- Filtros:
 - Por canal (WhatsApp, Web, Instagram, etc.)
 - Por estado (Activa, Resuelta, Bot)
 - Por fecha (Hoy, Ayer, Última semana)
- Búsqueda por nombre o teléfono

DETALLE DE CONVERSACIÓN:

Al hacer click en una conversación, se abre el chat completo:

Header:

- Nombre del cliente
- Canal (icono de WhatsApp, Web, etc.)
- Estado: "Bot atendiendo" / "Humano atendiendo"
- Botones:
 -  Transferir a bot
 -  Tomar control (transferir a humano)
 -  Marcar como resuelta
 -  Eliminar conversación

Mensajes:

- Timeline de mensajes (orden cronológico)
- Cada mensaje muestra:
 - Avatar del emisor (cliente o bot/humano)
 - Nombre del emisor
 - Timestamp
 - Contenido del mensaje (texto, imagen, audio, etc.)
 - Badge si fue enviado por bot o humano
 - Estado del mensaje (Enviado, Entregado, Leído)
- Mensajes del cliente: alineados a la izquierda
- Mensajes del bot/humano: alineados a la derecha
- Indicador de "Bot está escribiendo..." si el bot está procesando

Input de Mensaje:

- Textarea para escribir mensaje manual
- Botón "Enviar"
- Botones de respuestas rápidas (predefinidas)
- Adjuntar archivos (imágenes, PDFs)
- Emojis
- Si está en modo bot: muestra "Modo Bot Activado" y no se puede escribir manualmente

Panel Lateral (Información del Cliente):

- Datos del cliente (nombre, teléfono, email)
- Historial de reservas
- Última reserva
- Total de reservas
- Notas
- Etiquetas

ACCIONES:

- Transferir a Bot: El bot continúa la conversación automáticamente
- Tomar Control: Un humano toma la conversación (el bot se silencia)
- Marcar como Resuelta: Cierra la conversación
- Enviar Mensaje Manual: Escribe y envía mensaje como humano
- Usar Respuesta Rápida: Inserta mensaje predefinido
- Ver Transcripción Completa: Exporta la conversación a PDF

ESTADÍSTICAS DE AGENTES IA:

Panel superior con métricas:

- Conversaciones totales (hoy/semana/mes)
- Conversaciones atendidas por bot (%)
- Conversaciones transferidas a humano (%)
- Tiempo promedio de respuesta del bot
- Tasa de resolución del bot (% de conversaciones resueltas sin humano)
- Reservas generadas por bot

REQUISITOS TÉCNICOS:

Componentes:

```
typescript
// ConversationsList.tsx
interface ConversationsListProps {
  conversations: Conversation[];
  onSelectConversation: (conversationId: string) => void;
  selectedConversationId?: string;
}

// ConversationDetail.tsx
interface ConversationDetailProps {
  conversation: Conversation;
  onSendMessage: (message: string) => void;
  onTransferToBot: () => void;
  onTakeControl: () => void;
  onMarkResolved: () => void;
}
```

```

// MessageBubble.tsx
interface MessageBubbleProps {
  message: Message;
  isFromCustomer: boolean;
}

// MessageInput.tsx
interface MessageInputProps {
  onSend: (message: string) => void;
  disabled?: boolean;
}

// QuickReplies.tsx
interface QuickRepliesProps {
  replies: string[];
  onSelect: (reply: string) => void;
}

// CustomerInfoPanel.tsx
interface CustomerInfoPanelProps {
  customer: Customer;
  reservations: Booking[];
}

// ConversationFilters.tsx
interface ConversationFiltersProps {
  filters: ConversationFilters;
  onFilterChange: (filters: ConversationFilters) => void;
}

// BotStats.tsx
interface BotStatsProps {
  stats: BotStats;
}

```

Real-time con [Socket.io](#):

```

typescript
// server/socket.ts
io.on('connection', (socket) => {
  socket.on('join-conversation', (conversationId: string) => {
    socket.join(`conversation-${conversationId}`);
  });
});

```

```

    socket.on('send-message', async (data: { conversationId:
string, message: string }) => {
        const message = await saveMessage(data);

io.to(`conversation-${data.conversationId}`).emit('new-message
', message);
    });

    // Cuando el bot responde
    socket.on('bot-response', (data: { conversationId: string,
message: string }) => {

io.to(`conversation-${data.conversationId}`).emit('new-message
', {
    ...data,
    sender: 'bot',
});
});
});

// client/hooks/useConversation.ts
export function useConversation(conversationId: string) {
    const [messages, setMessages] = useState<Message[]>([]);

    useEffect(() => {
        const socket = io(process.env.NEXT_PUBLIC_WS_URL!);

        socket.emit('join-conversation', conversationId);

        socket.on('new-message', (message: Message) => {
            setMessages(prev => [...prev, message]);
        });

        return () => {
            socket.disconnect();
        };
    }, [conversationId]);

    const sendMessage = (text: string) => {
        socket.emit('send-message', { conversationId, message:
text });
    };

    return { messages, sendMessage };
}

```

```
}
```

API Endpoints:

typescript

```
// GET /api/conversations
```

Query params:

- restaurantId: string
- channel?: string
- status?: string
- search?: string
- page?: number
- pageSize?: number

```
Response: {  
  conversations: Conversation[],  
  total: number,  
}
```

```
// GET /api/conversations/{id}
```

```
Response: {  
  conversation: Conversation,  
  messages: Message[],  
  customer: Customer,  
}
```

```
// POST /api/conversations/{id}/messages
```

```
Body: {  
  message: string,  
  sender: 'human' | 'bot',  
}
```

Response: Message

```
// POST /api/conversations/{id}/transfer-to-bot
```

```
Response: { success: boolean }
```

```
// POST /api/conversations/{id}/take-control
```

```
Response: { success: boolean }
```

```
// POST /api/conversations/{id}/resolve
```

```
Response: { success: boolean }
```

```
// GET /api/conversations/stats
```

Query params:

- restaurantId: string
- startDate: YYYY-MM-DD

```
- endDate: YYYY-MM-DD
```

```
Response: {  
  totalConversations: number,  
  botHandled: number,  
  humanHandled: number,  
  avgResponseTime: number,  
  resolutionRate: number,  
  bookingsGenerated: number,  
}
```

Database Schema:

```
prisma  
model Conversation {  
  id          String    @id @default(uuid())  
  restaurantId String  
  customerId  String?  
  channel     String    // whatsapp, web, instagram,  
facebook  
  status      String    @default("active") // active,  
resolved, bot  
  assignedTo  String?   // userId del humano asignado  
  lastMessageAt DateTime @default(now())  
  createdAt   DateTime @default(now())  
  updatedAt   DateTime @updatedAt  
  
  restaurant Restaurant @relation(fields: [restaurantId],  
references: [id])  
  customer   Customer?  @relation(fields: [customerId],  
references: [id])  
  messages   Message[]  
  
  @@index([restaurantId, status])  
}  
  
model Message {  
  id          String    @id @default(uuid())  
  conversationId String  
  sender      String    // customer, bot, human  
  senderId    String?   // userId si es humano  
  content     String  
  messageType String    @default("text") // text, image,  
audio, file  
  mediaUrl    String?
```

```
status      String    @default("sent") // sent,
delivered, read

createdAt   DateTime  @default(now())

conversation Conversation @relation(fields:
[conversationId], references: [id], onDelete: Cascade)

@@index([conversationId, createdAt])
}
```

UI/UX:

- Layout de 2 columnas: lista de conversaciones (izquierda) + detalle (derecha)
- Lista de conversaciones con scroll
- Detalle con header, mensajes y input
- Mensajes con burbujas de chat (estilo WhatsApp)
- Avatares con iniciales o fotos
- Timestamps en formato relativo ("Hace 5 min")
- Indicador de "escribiendo..."
- Badges de no leídos
- Colores por canal (verde WhatsApp, azul Web, etc.)
- Animaciones suaves al recibir mensajes nuevos
- Responsive: en móvil, lista y detalle son pantallas separadas
- Accesibilidad: ARIA labels, keyboard navigation

RESUMEN DE ENTREGABLES ESPERADOS

Después de implementar este módulo, debes tener:



Dashboard Overview:

- Métricas clave en tiempo real
- Gráfico de ocupación por hora
- Lista de próximas reservas
- Panel de alertas y notificaciones
- Actualizaciones en tiempo real (WebSockets)



Gestión de Reservas:

- Vista de calendario (día/semana/mes)
- Vista de lista con filtros avanzados
- Vista de floor plan (plano de mesas)

- Modal de detalles de reserva
- Acciones rápidas (check-in, no-show, editar, cancelar)
- Drag & drop para mover reservas
- Exportación a Excel/PDF

✓ CRM de Clientes:

- Lista de clientes con filtros
- Perfil completo del cliente
- Historial de reservas
- Preferencias y etiquetas
- Segmentación automática (VIP, Regular, Nuevo, Inactivo)
- Historial de comunicaciones
- Acciones masivas

✓ Analytics:

- KPIs principales
- Gráficos interactivos (reservas, ocupación, origen, etc.)
- Heatmap de ocupación
- Insights automáticos
- Exportación de reportes
- Filtros por fecha y sucursal

✓ Configuración:

- Gestión de restaurante y sucursales
- Configuración de horarios y mesas
- Configuración de pagos y pasarelas
- Gestión de personal y permisos
- Integraciones (POS, Google, Webhooks)
- Notificaciones y políticas

✓ Mensajes (Agentes IA):

- Lista de conversaciones
- Chat en tiempo real
- Transferencia bot ↔ humano
- Respuestas rápidas
- Estadísticas de agentes IA
- Información del cliente en panel lateral

✓ UI/UX:

- Diseño responsive (mobile-first)
- Dark mode support
- Accesibilidad WCAG 2.1 AA
- Animaciones suaves
- Loading states y skeleton loaders
- Error handling robusto
- Notificaciones toast

✓ Real-time:

- WebSockets para actualizaciones en tiempo real
- Notificaciones push
- Sincronización entre múltiples usuarios

✓ Testing:

- Tests unitarios de componentes
- Tests de integración de flujos
- Tests E2E con Playwright

INSTRUCCIONES PARA ANTIGRAVITY

1. Comienza con la estructura de navegación (sidebar, routing)
2. Implementa el sistema de autenticación y roles
3. Desarrolla el Overview (métricas y gráficos básicos)
4. Construye la gestión de reservas (calendario, lista, floor plan)
5. Implementa el CRM de clientes
6. Agrega el dashboard de analytics
7. Crea el panel de configuración
8. Implementa el módulo de mensajes (agentes IA)
9. Agrega real-time con WebSockets
10. Optimiza y asegura la calidad (tests, performance, accesibilidad)

Prioridad:

1. Gestión de reservas (core del negocio)
2. Overview (dashboard principal)
3. Configuración básica
4. CRM de clientes

- 5. Analytics
- 6. Mensajes (agentes IA)
- 7. Características avanzadas

Nota: Cada sección debe ser funcional independientemente, pero integrarse perfectamente con las demás. El dashboard debe ser rápido y responsivo, ya que se usa durante el servicio del restaurante.

¿Listo para comenzar? Empieza por la estructura de navegación y el sistema de autenticación, luego avanza al Overview y la gestión de reservas.